

DeliveryTracker — System Design Document

1. Introduction

DeliveryTracker is a last-mile delivery management platform that addresses three interconnected logistics problems: computing an accurate, non-negotiable delivery charge at the moment of booking; assigning the shipment to a delivery agent without manual dispatcher intervention; and keeping the customer informed through every state change, including failure and recovery. The platform supports three roles — Customer, Delivery Agent, and Admin — covering the full order lifecycle from creation through pricing, assignment, live tracking, and delivery or failed-delivery rescheduling.

2. Overall Architecture

The system is split into a Spring Boot 3 (Java 21) REST backend and a React + Vite single-page frontend, communicating over a JSON API secured with JWT bearer tokens. Data is persisted in PostgreSQL via Spring Data JPA. The backend follows a feature-based modular architecture — `auth`, `shipment`, `pricing`, `assignment`, `tracking`, `notification`, and `admin` are independent packages, each owning its own entities, repositories, services, controllers, and exceptions, rather than being split by technical layer, keeping each domain self-contained and testable in isolation.

Side effects — email/SMS notifications — are not called directly from business logic. Services instead publish Spring `ApplicationEvent`s (`ShipmentCreatedEvent`, `PriceCalculatedEvent`, `AssignmentCreatedEvent`, `TrackingUpdatedEvent`), and a `NotificationEventListener` reacts to them, decoupling order processing from notification delivery.

For deployment, the backend is containerized with a multi-stage `Dockerfile` and orchestrated via `docker-compose.yml` alongside PostgreSQL, with GitHub Actions running the Maven test suite on every merge; it is deployed live on Railway at `https://deliverytracker-production-507b.up.railway.app`. The frontend, built with Vite and configured with a `vercel.json` SPA rewrite rule, is deployed on Vercel at `https://deliverytracker.vercel.app`.

3. Zone Detection

Zones are admin-defined entities. Each `Area` record maps a pincode to exactly one `Zone` and stores latitude/longitude for that area. On order creation, the pickup and delivery pincodes are each resolved to their `Area` via `AreaRepository.findByPincode()`, and each `Area` exposes its parent `Zone`. If a pincode has no matching `Area`, an `AreaNotFoundException` is thrown immediately rather than defaulting silently, since an unmapped pincode has no defined rate relationship.

4. Rate Calculation Engine

The pricing flow, implemented in `PricingServiceImpl` and `PricingCalculator`, follows this sequence: Pickup Pincode → Pickup Area → Pickup Zone, and Delivery Pincode → Delivery Area → Delivery Zone. Volumetric weight is $(\text{Length} \times \text{Width} \times \text{Height}) / 5000$. Billable weight is $\max(\text{actualWeight}, \text{volumetricWeight})$. This weight, the zone pair, and order type are used to look up the matching `RateCard` via

`RateCardRepository.findRateCard(pickupZone, deliveryZone, orderType, billableWeight)`, separately for B2B and B2C. The final charge is $\text{baseCharge} + \text{weightCharge} (\text{billableWeight} \times \text{pricePerKg}) + \text{fuelCharge} (\% \text{ of base}) + \text{codCharge} (\text{flat, if COD})$, with GST applied on that subtotal. Every rate card field — base charge, per-kg rate, fuel %, COD charge, GST % — is admin-configurable via `AdminController`'s rate card CRUD endpoints; no pricing value is hardcoded.

5. Auto Assignment

Assignment uses the Strategy pattern. `AssignmentServiceImpl` orchestrates the workflow: it fetches all `DeliveryAgent` records with `AvailabilityStatus.AVAILABLE`, then delegates candidate selection to `NearestAgentStrategy`, which calls `DistanceCalculator` to compute Euclidean distance between the pickup area's coordinates and each candidate's current coordinates, returning the agent with the minimum distance. The selected agent's availability flips to `BUSY`, an `OrderAssignment` record is created, and an `AssignmentCreatedEvent` is published. Admins can override auto-assignment via a manual assignment endpoint.

6. Tracking

Every status transition — creation, assignment, pickup, transit, delivery, or failure — is recorded as a new, append-only `TrackingHistory` row rather than an update to a mutable field, capturing status, location, remarks, actor, and timestamp. Rows are never edited, so the full timeline is reconstructed by querying all entries for a shipment ordered by time, giving an auditable, chronological delivery journey.

7. Failed Delivery Workflow

When an agent marks a shipment `FAILED`, that status is logged to `TrackingHistory` and a `TrackingUpdatedEvent` triggers a customer notification.

`ShipmentServiceImpl.rescheduleShipment` only accepts shipments currently `FAILED`, resets the status to `CREATED`, logs a customer-attributed reschedule entry, and re-invokes `assignShipment`, running the same nearest-agent selection from scratch for the new attempt.

8. Notification System

`EmailService` sends transactional emails via `JavaMailSender`, logging a warning if dispatch fails so a mail outage never breaks the request. `SmsService` currently logs the outbound SMS content rather than calling an external provider. Both are invoked only from `NotificationEventListener`, subscribed to events from the shipment, pricing, and assignment services.

9. Security

Authentication is stateless JWT-based: `JwtServiceImpl` issues signed tokens on login, and `JwtAuthenticationFilter` validates them on every request. `SecurityConfig` enforces `SessionCreationPolicy.STATELESS` and permits only `/api/v1/auth/**` and Swagger routes without a token. Role-based access control uses the `Role` enum (`ADMIN`, `CUSTOMER`, `AGENT`) with `@PreAuthorize("hasRole('ADMIN')")` on admin endpoints.

10. Conclusion

The implemented architecture satisfies the assignment's core requirements: a fully admin-configurable, no-hardcoding rate engine; data-driven zone detection; a pluggable nearest-agent assignment strategy; an immutable, auditable tracking timeline; and an event-driven notification layer covering the failed-delivery-to-reassignment loop end to end.